

NAMASTE JAVASCRIPT

-by AkashSaini

<https://www.youtube.com/watch?v=pN6jk0uUrD8&t=20s>

- ◆ Everything in java script happens (executed) inside **Execution Context**.
- ◆ Execution Context has 2 components:
 - Memory Component (also known as variable Enviroment) – all variables and functions are stored in key value pairs.
 - Code Component (also Known as Thread of Execution) – here code is executed one line at a time.
- ◆ Java Script is a **synchronous, single-threaded** language(mean can execute one command at a time with specific order i.e it can go to next line only by executing the current line of code).
- ◆ When we run a js program a global execution engine is created.
- ◆ Execution Context is created in 2 phases:
 - Memory Creation Phase
 - Code Execution Phase
- ◆ Execution Context for the below program:

```
var n=2;
function square(num){
    var ans = num * num;
    return ans;
}
var square2 = square(n);
var square4 = square(4);
```

Memory Creation Phase:

Memory	Code
N : Undefined	
Square : {.....}(whole code of function is stored in this memory space)	
Square2: Undefined	
Square4: Undefined	

Code Execution Phase:

Memory	Code												
N : Undefined 2	N = 2												
Square : {.....}													
Square2: Undefined 4	Memory Creation Phase: <table border="1"> <thead> <tr> <th>Memory</th><th>Code</th></tr> </thead> <tbody> <tr> <td>num :Undefined</td><td></td></tr> <tr> <td>ans: Undefined</td><td></td></tr> </tbody> </table> Code Execution Phase: <table border="1"> <thead> <tr> <th>Memory</th><th>Code</th></tr> </thead> <tbody> <tr> <td>Num : 2</td><td>ans = 2 * 2 = 4</td></tr> <tr> <td>ans: 4</td><td>Return ans</td></tr> </tbody> </table> This Execution Context is deleted after returning the value.	Memory	Code	num :Undefined		ans: Undefined		Memory	Code	Num : 2	ans = 2 * 2 = 4	ans: 4	Return ans
Memory	Code												
num :Undefined													
ans: Undefined													
Memory	Code												
Num : 2	ans = 2 * 2 = 4												
ans: 4	Return ans												
Square4: Undefined	Execution Context is created similarly for Square4.												

- ◆ After the Execution whole global Execution Context is deleted.
- ◆ Functions are heart of javascript.They work differently from any other programming languages.
- ◆ Example for **Hoisting**:
Hoisting is a phenomena in which you can access the variables and functions before their initialization.

program:

```

getName();
console.log(x);

var x = 7;
function getName(){
    console.log("NAMASTE JavaScript");
}

```

Output:

```

NAMASTE JAVASCRIPT
Undefined

```

program:

```
getName();  
console.log(x);
```

```
function getName(){  
    console.log("NAMASTE JavaScript");  
}
```

Output:

ReferenceError: x is not defined

- ◆ If we access a variable before initialization we get the output as Undefined but in case of function it prints the function. This is because in javascript even before the code starts execution javascript allocates memory to all the variables and functions where functions are stored with their definition. (memory creation phase)
- ◆ if we have getName as the arrow function then getName acts as another variable so when we write console.log(getName) we will get Undefined as output.

program:

```
console.log(getName());  
console.log(getName);  
console.log(x);
```

```
var x = 7;  
var getName = () => {  
    console.log("NAMASTE JavaScript");  
}
```

output:

TypeError: getName is not a function

- ◆ We can also create a function as below:
Even here getName just behaves like a variable.
Only in case of proper functions javascript allocates memory to function along with their definition at memory creation phase.

program:

```
console.log(getName());  
console.log(getName);  
console.log(x);
```

```
var x = 7;  
var getName = function () {  
    console.log("NAMASTE JavaScript");  
}
```

```
}
```

Output:

TypeError: getName is not a function

- ◆ Even the js file is empty but still java script engine will do its job by creating global executing context and setting up the memory space even there is nothing to set up. JS Engine also creates a Window(global object created along with global execution context), this keyword (At global level this refers to window object) (this is also created along with global Execution Context).
- ◆ Window contains lot of functions and methods created by JSEngine added to the global space.
- ◆ **Global Space** anything written on top-level or anything that is not written inside the functions.
- ◆ In JavaScript, undefined is a type, whereas null is an object. **Undefined** means a variable declared, but no value has been assigned a value. Whereas, **null** in JavaScript is an assignment value. You can assign it to a variable.
- ◆ JavaScript is a loosely typed language.

Program:

```
var a;  
console.log(a);  
a=10;  
console.log(a);  
a="Hello world";  
console.log(a);  
a =undefined; //not a mistake but not a good practise  
console.log(a)
```

Output:

```
undefined  
10  
Hello World  
undefined
```

- ◆ **Scope** in javascript is directly related to lexical environment.
- ◆ Global execution context is created and put on to the call stack.

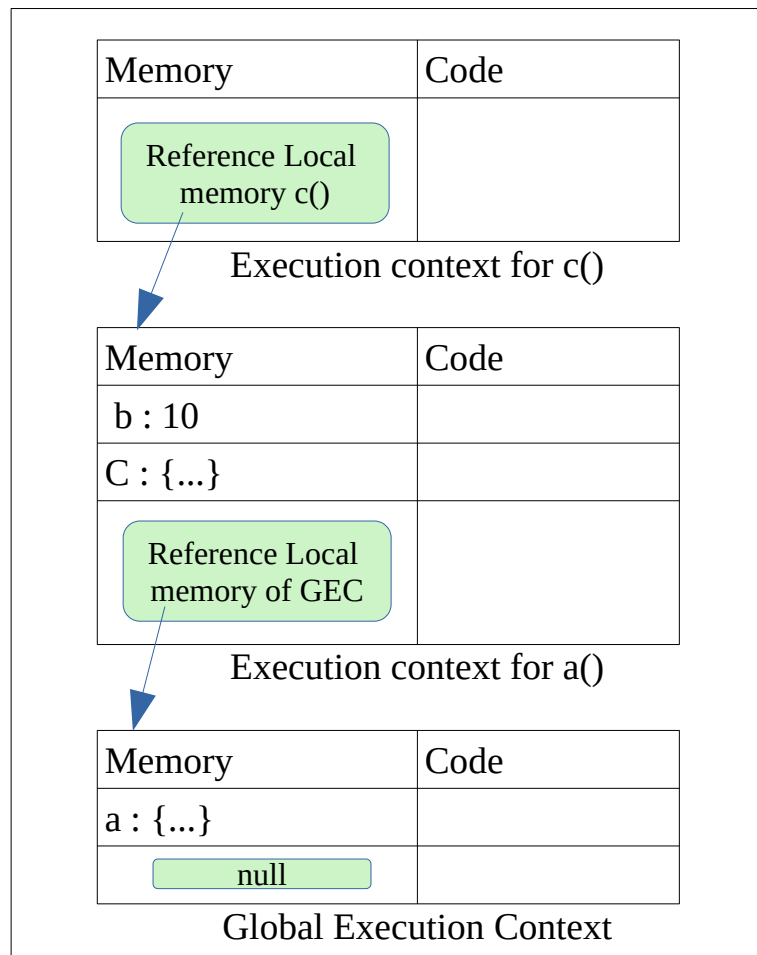
Program:

```
function a(){  
    var b = 10;  
    c();  
    function c(){  
  
    }  
}
```

```
a();
console.log(b);
```

Output:
ReferenceError: b is not defined

Call stack for above program:



- ◆ Where ever a execution context is created a lexical environment is also created.
- ◆ Lexical environment is a local memory with the lexical environment of its parent.
- ◆ Function “C” is physically present inside Function “A” which means “C” is present lexically inside “A”.

Lexical environment = local memory + memory of its parent

- ◆ Example with let and var keywords:
 - In the below program if we write console.log(b) instead of console.log(a) it returns undefined but returns error when we access a.
 - This is because variables declared with var are attached to global object (memory) where as variables with const and let are allocated to different memory space(called Script) which doesn't allow you access variables without value in them.

- The phase from point of hoisting to the time until a variable is initialized is known as a “ **Temporal Dead Zone** ”.i.e in below code ‘a’ is hoisted at line 1 and initialized at line 2.

Whenever we access a variable in temporal dead zone it returns a ReferenceError.i.e any code before line 2 is a temporal dead zone for ‘a’.

Program:

```
console.log(a);
let a=10;
var b=100;
```

Output:

ReferenceError: cannot access ‘a’ before initialization

- The let and const variables cannot be accessed with window or this objects where var variables can access(this is because var variables are attached to window object).

Program:

```
let a=10;
var b=100;
window.b;
window.a;
this.b;
this.a;
```

Output:

```
100
undefined
100
undefined
```

- let is more strict than var.

Program:

```
let a=10;
var b=100;
var b =1000;
```

This code doesn't give any error where as if we do same with let it raises an error.

Program:

```
let a=10;
let b=100;
```

```
let b =1000;
```

Output:

SyntaxError : Identifier 'b' has already been declared.

- Const is little more strict than let.

Program:

```
let a;
```

```
let b=100;
```

```
let b =1000;
```

Output:

SyntaxError : Identifier 'b' has already been declared.

Using let we can declare at one place and initialize somewhere in the program but this is not possible with const.

Program:

```
let a;
```

```
a=100;
```

```
const b;
```

```
b =1000;
```

Output:

SyntaxError : Missing Initializer in const declaration.

Program:

```
const b=1000;
```

```
b =10;
```

Output:

TypeError : Assignment to constant variable

◆ **Block Scope**

- block is also known as compound statement (we group multiple statements together in a place so that it can be used where js expects one statement for this we use block)
- Block Scope means what all the variables we can access inside this block.
- var 'a' is in global scope where as 'b' and 'c' are hoisted in separate memory (i.e block scope). This is why we hear let and const are block scoped.
we cannot access these let and var outside the block but we can access variable with var outside the block too.

◆ **Shadowing**

- if we have a same named variable outside the block and inside the block, the local variable shadows the global variable.
- In case of var

Program:

```
var a=100;
{
    var a = 10;//shadowing by changing 100 to 10
    let b = 20;
    const c = 30;
    console.log(a); //10
    console.log(b); //20
    console.log(c); //30
}
console.log(a);//10
```

- In case of let

Program:

```
let b=100;//global
{
    var a = 10;//local
    let b = 20;//local
    const c = 30;//local
    console.log(a); //10
    console.log(b); //20
    console.log(c); //30
}
console.log(b);//100
```

- In case of Const

Program:

```
const c = 100;//global
{
    var a = 10;//local
    let b = 20;//local
    const c = 30;//local
    console.log(a); //10
    console.log(b); //20
    console.log(c); //30
}
console.log(c);//100
```

- ◆ **Illegal Shadowing**

Program:

```
let a = 20;
{
    var a = 20;//illegal shadowing
}
console.log(a);
```

Output:

SyntaxError: identifier 'a' is already declared

Program:

```
var a = 20;
{
  let a = 20;
}
console.log(a);
```

But this way js doesn't throw an error. As var is (global) function scoped.

Program:

```
let a = 20;
{
  let a = 20;
}
console.log(a);
```

Even this works and known as shadowing. (Each and every block has its own lexical scope.)

- ◆ **Closures** : Function along with its lexical scope forms a closure.

or

Closure is the combination of Function bundled together with references of surrounding state (lexical environment)

Program:

```
function x(){
  var a = 7;
  function y(){
    console.log(a);
  }
  y();
}
x();
```

Output:

7

In this program if we inspect we get closure(x) as a:7

Program:

```
function x(){
  var a = 7;
  function y(){
    console.log(a);
  }
  return y;
}
var z = x();
console.log(z);
z();//7
```

Output:

```
f: y(){
  console.log(a);
}
7
```

This is possible in javascript even if it is not variable it returns the function. In this above program as x completed its execution it is removed from GEC But z(); return value 7 as output that is because of closures. Though x no longer exists y still remembers its lexical scope and where it came from. Not only y is returned when called x() but closure of x (function of y +lexical scope) is returned.

Program:

```
function x(){
  var a = 7;
  function y(){
    console.log(a);
  }
  a = 100;
  return y;
}
var z = x();
z();
```

Output:

```
100
```

function y() remembers the reference of a not the value so now, as 100 is updated in memory for a the closure(x) will be a:100.

Program:

```
function z(){
  var b = 100;
  function x(){
    var a = 7;
    function(y){
      console.log(a,b);
    }
    y();
  }
  x();
}
z();
```

For this program:

```
Closure (x) : a : 7
Closure(z): b:100
```

Output:

```
7 100
```

◆ Uses of Closures:

- Module Design Patterns
- Currying

- Functions like once
 - memoize
 - maintaining state in async world
 - setTimeouts
 - Iterators
- ◆ **setTimeout** function takes the call back function and attaches the timer to it and stores somewhere and then goes to next line and executes. when the timer expires takes the function again to call stack and executes.

Program:

```
function x(){
  var a = 7;
  setTimeout(function (){
    console.log(a);
  },3000);
  console.log("Namaste JavaScript");
}
x();
```

Output:

Namaste JavaScript
7

Program:

```
function x(){
  for(var i =1;i<=5;i++){
    setTimeout(function (){
      console.log(i);
    },i*1000);
  }
  console.log("Namaste Javascript");
}
x();
```

Output:

Namaste JavaScript
6
6
6
6
6

The above program displays this output because of closures. Even if the function is taken out from its original scope and stored somewhere and attaches the timer it remembers (the lexical scope) reference of i. And JS Engine doesn't wait for anything so the loop continues running. So by the time timer expires 'i' has the value of 6.

Easy way to fix it is to change var to let for i as let is block scoped.

Program:

```
function x(){
  for(let i =1;i<=5;i++){
    setTimeout(function (){
      console.log(i);
    },i*1000);
  }
  console.log("Namaste Javascript");
}
x();
```

Output:

```
Namaste JavaScript
1
2
3
4
5
```

Here whenever the loop is repeated, setTimeout method is called and this function forms closure with new variable. And everytime for i it is referncing to a different memory location.

Another way to fix using var keyword:

Easy way to fix it is to change var to let for i as let is block scoped.

Program:

```
function x(){
  for(var i =1;i<=5;i++){
    function close(){
      setTimeout(function (){
        console.log(i);
      },i*1000);
    }
    close(i);
  }
  console.log("Namaste Javascript");
}
x();
```

Output:

```
Namaste JavaScript
1
2
3
4
5
```

Here whenever the close() function is called with 'i' it creates a new copy of i for itself at function definition.

- ◆ Closure helps with **data hiding**.

If we generally write `count=0` and `count++` anyone can modify the value but when we wrap it within a function due to closures we couldn't access it from outside.

Program:

```
function counter(){
    var count =0;
    return function incrementCounter(){
        count++;
    }
}
console.log(count);
```

Output:

ReferenceError : count is not defined

- ◆ **Constructor function** for a constructor function we need to call it using `new` keyword.

Program:

```
function Counter(){
    var count =0;
    this.incrementCounter = function(){
        count++;
        console.log(count);
    }
    this.decrementCounter = function(){
        count--;
        console.log(count);
    }
}
var counter1 = new Counter();
counter1.incrementCounter();
counter1.decrementCounter();
```

Output:

1
0

- ◆ **Disadvantages of closures:**

- Over Consumption of memory
- Sometimes those closed over variables are not garbage collected
- if not handled properly that may lead to memory leaks.

- ◆ **Function Statement:**

```
function a(){
    console.log("This is a");
}
```

- ◆ **Function Expression:**

```
var b = function (){  
    console.log("This is b");  
}
```

- ◆ The difference between function statement and function expression is hoisting. We couldn't call the function expression before creation where we can call function statement before creation.
- ◆ Function statement or function declaration is the same.
- ◆ Anonymous Function : the functions without a name is known as anonymous functions. These can be used when functions are used as values. These cannot be written just like function statements. In that case it returns syntax error

```
var a = function(){  
}
```

- ◆ **Named function Expression:**

Named function Expression is same as the function expression but instead of using anonymous function to write a block of code we use name for a function. But in this case if we call xyz(); it gives error as this xyz is written as value and created as a local variable.

```
var b = function xyz(){  
    console.log("This is b");  
}
```

we could access the xyz inside the function

```
var b = function xyz(){  
    console.log(xyz);  
}
```

- ◆ **First Class Functions: or First Class Citizens**

Functions can be passed as arguments to a function (passing a function inside the function and returning a function from the function) .

The ability to use functions as values is known as first class functions.

```
var b = function (param 1){  
    return function xyz(){  
    }  
}
```

- ◆ **Call Back Functions**

Java Script is a synchronous single threaded language but due to call back functions we can do asynchronous things.

if we call a function and pass another function the function passed is called as call back function. As we give the responsibility of this function to another function to call it after sometime it is known as call back function.

```
function x(y){  
  
    }  
    x(function y(){  
  
    })
```

here y is known as call back function. Best Example setTimeout function.

- ◆ Javascript has only one call stack. Everything executes through the call stack. blocking the call stack means blocking the main thread. This is why we should not block main thread (call stack). That is why we should prefer to use asynchronous operations for things that take time.

- ◆ **Event Listeners:**

```
document.getElementById("click").addEventListener("click",function  
    xyz(){  
        console.log("Button Clicked");  
    });
```

when ever we click a button the already stored call back function is brought back and executed.

- ◆ **Closures with Event listeners:**

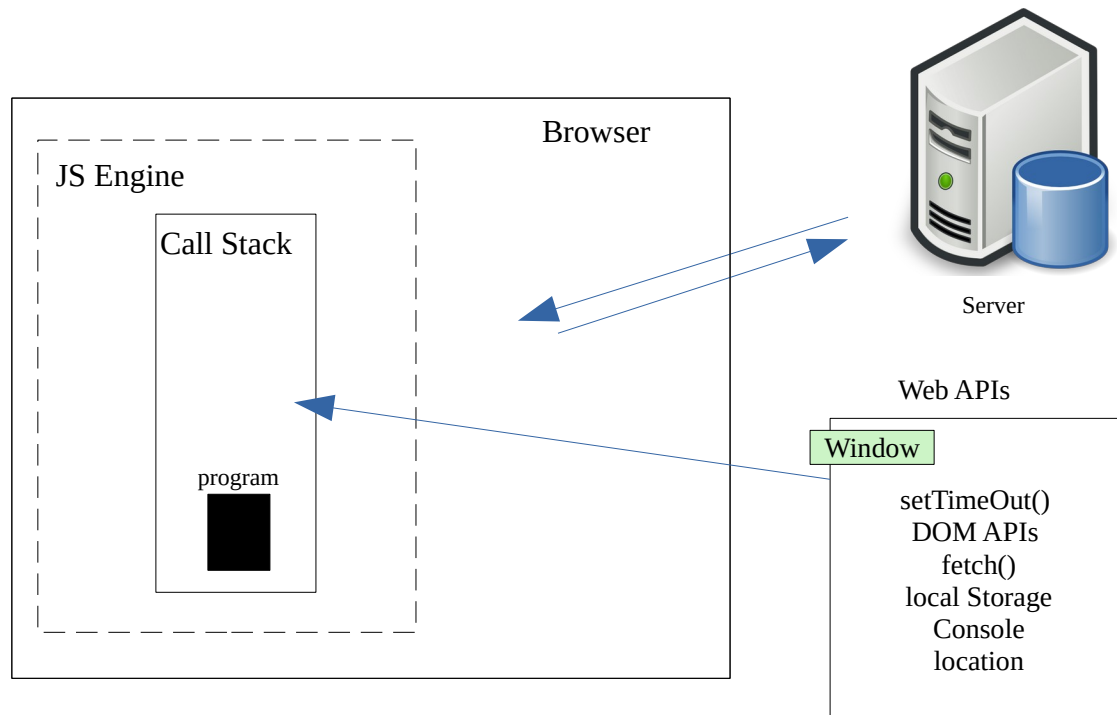
```
function attachEventListener(){  
    let count = 0;  
    document.getElementById("click").addEventListener("click",function(){  
        console.log("Button Clicked",++count);  
    });  
}  
attachEventListener();
```

Here the above count variable is block scoped because of let variable, so it gets updated every time the button is clicked.

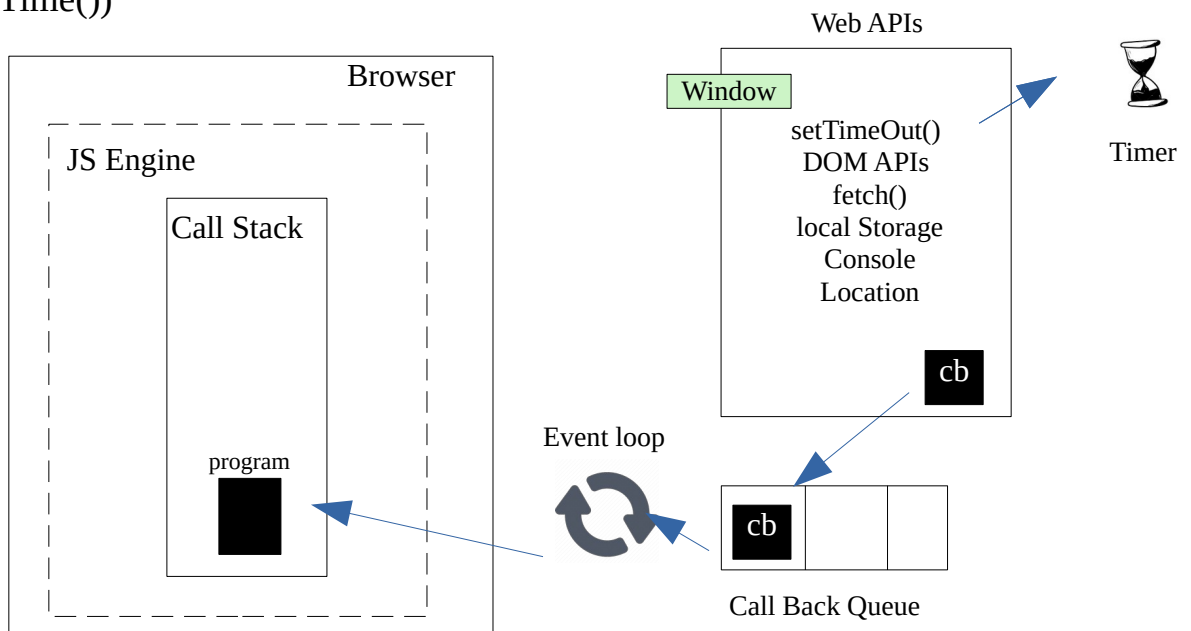
- ◆ **Garbage Collection and remove Event Listeners**

Event Listeners are heavy. This main reason why we need to remove Event Listeners. Because of so many closures, call back functions our page can go lot of slow. so it's a good practice to remove Event Listeners, then all the variables which were held by Closures are garbage collected and frees up the memory.

- ◆ Java Script doesn't wait for anything and when the code comes into call stack it executes immediately. if we want to execute the code after 5 sec it is not possible in call stack as call stack has no timer.
- ◆ Browser is super powerful. It has timer inside it. Browser has the power to communicate with external world.



- ◆ To make browser that powerful we need some web APIs. And these web APIs of browser are accessed by call Stack.
- ◆ Browser gives js Engine the facility to use this super powers of timer through keyword window. Ex: `window.setTimeout()`; But as window is a global object so we can access these functions with or without window keyword. (i.e `setTimeout()`)



Program:

```
setTimeout(function cb(){
    console.log("Callback");
},5000);
```


In above program the setTimeout method of browser invokes the timer and the call back function cb is stored in a place and at later point when the timer expires the cb function is pushed back on to callback queue. The event loop checks whether there is anything in call back queue. if there presents anything on call back queue it sends it to call stack where the code gets quickly executed.

◆ Event Loop

Event loop acts as gatekeeper and checks whether something is present inside the call back queue and pushes back to call stack if anything present in call back queue.

Event Loop has only one job continuously monitor the call stack and call back queue.

◆ Why do we need a call Back Queue?

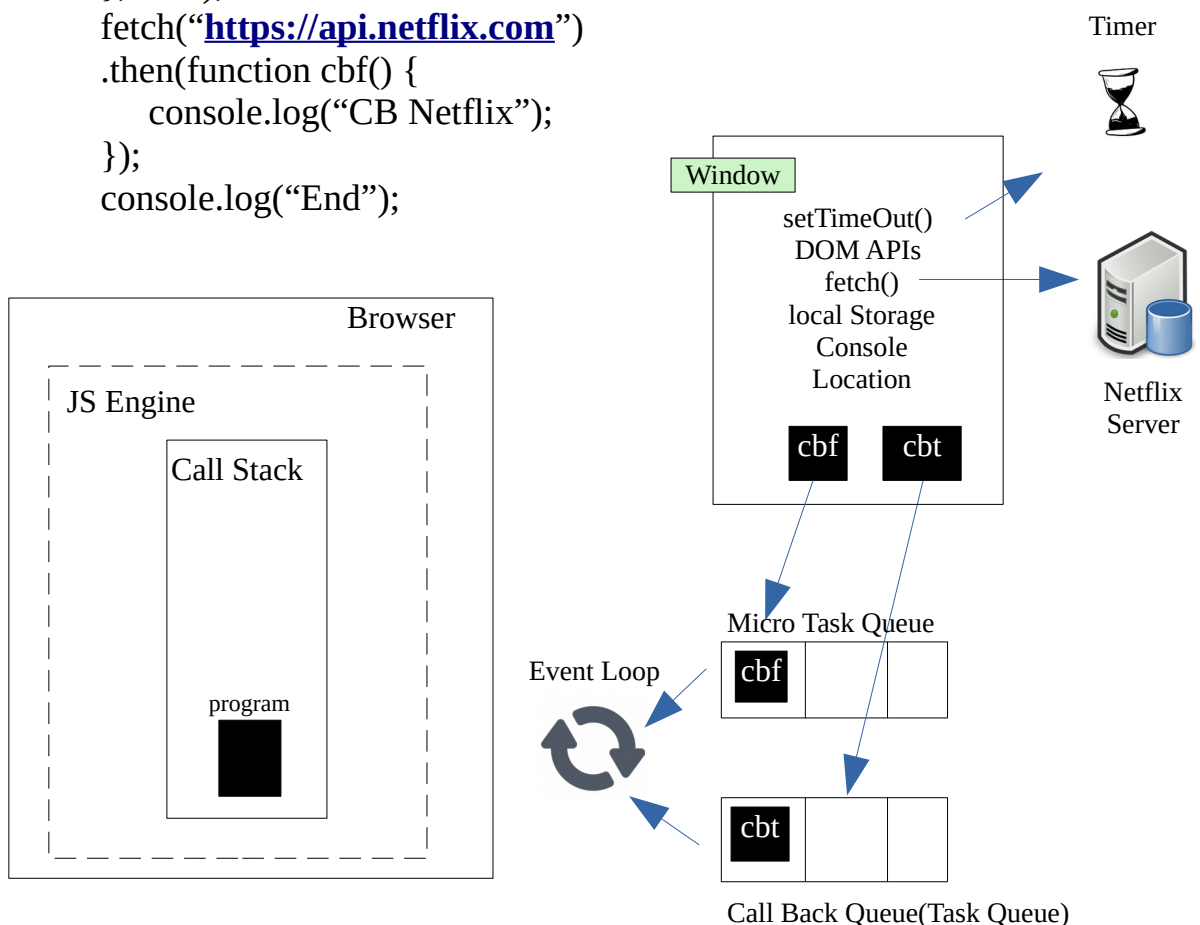
If we click a button multiple times while using event listeners then all the callback functions cannot be sent to call stack as call stack can execute only one by one (single threaded). So every time a button is clicked those many times the function is pushed to the callback queue.

◆ fetch() basically goes and requests a api call.

Fetch returns a promise and call back function is executed once the promise is resolved.

Program:

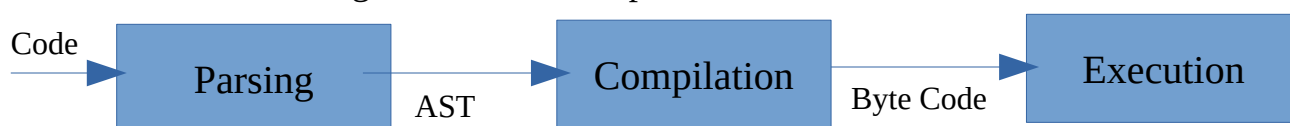
```
setTimeout(function cbt(){
  console.log("CB setTimeout");
},5000);
fetch("https://api.netflix.com")
.then(function cbf() {
  console.log("CB Netflix");
});
console.log("End");
```



Micro Task Queue has High priority than call back queue.for fetch() function once the data is returned the call back function cbf is is pushed on to microtask queue which has high priority.The event loop continuously monitors the call stack,callback queue,micro task queue.if the call stack is empty and micro task queue has anything presentwithin it then it gets pushed to call stack first and executed immediately.Later when callstack & micro Task queue is empty then things in call back queue are pushed to call stack and executed.

- ◆ There is a mutation observer that keeps on checking whether there is a mutation or not in the DOM tree,if there exists a mutation it can execute a callback function.
- ◆ All the call back functions comes through promises,as well as mutators Observer goes into microtask queue.
- ◆ JavaScript Engine is the heart of JavaScript Runtime Environment.
- ◆ Js Engine goes through:
code -> Parsing ->Compilation - >Execution
- ◆ **Parsing:**
 - broken down into tokens.
 - Syntax Parser takes the code and breaks down it into AST (Abstract Syntax Tree)(Ast is just like a dictionary format tree).
- ◆ **Compilation:**
 - Js have just in time Compilation
 - At the beginning js is an interpreted language.
 - But now it can be both interpreted and Compiled language that depends on JSEngine.
 - Compilation and Execution goes hand in hand.
 - JIT has Interpreter and compiler where the AST goes to Interpreter and converts high level language to byte code and that moves to Execution.
 - While interpreter is doing so,it takes help of compiler to optimize the code as much as it can at runtime.

In some Js Engine there is AOT(Ahead of Time Compilation) where compiler takes a piece of code which is going to be executed later and tries to optimize it and also produces byte code that goes to Execution phase.



- ◆ **Execution:**
 - Execution here is not possible with two main components:
 - ✓ **Memory Heap:** place where all the memory(variables and functions assigned memory) is stored.Memory heap is constantly in sync with callstack and Garbage collector.

✓ Call Stack

- ◆ Garbage collector uses an algorithm: **Mark and Sweep**.
- ◆ **Higher Order Functions** : A function which takes a function as an argument or returns a function as an argument is known as Higher order functions.

Program:

```
function x(){
    console.log("Namaste");
}
function y(x){
    x();
}
```

x is a call back function where y is higher ordered function.

Program:

```
let radius = [1,3,5]
const area = function (radius){
    return Math.PI*radius*radius;
}
const circumference = function (radius){
    return 2*Math.PI*radius;
}
const diameter = function (radius){
    return 2*radius;
}
const calculate = function(arr,logic){
    const output = [];
    for(let i=0;i<arr.length;i++){
        output.push(logic(arr[i]));
    }
    return output;
};
console.log(radius.map(area));
console.log(calculate(radius,area));
console.log(calculate(radius,circumference));
console.log(calculate(radius,calculate));
```

If we change "const" calculate" of calculate function to **"Array.prototype.calculate"** to make it work same as map. By doing so calculate will be available on all the arrays.

```
Array.prototype.calculate = function(logic){
    const output = [];
    for(let i=0;i<this.length;i++){
        output.push(logic(this[i])); }
}
```